

UNIVERSITY OF BOHOL
COLLEGE OF ENGINEERING, TECHNOLOGY, ARCHITECTURE AND FINE ARTS

A Compilation Report for

OPERATING SYSTEM CPEP312A

Final Project

Presented to the Dean and Faculty of

College of Engineering, Technology, Architecture, and Fine Arts

University of Bohol

Submitted to:

Engr. Liduvina L. Namocatcat

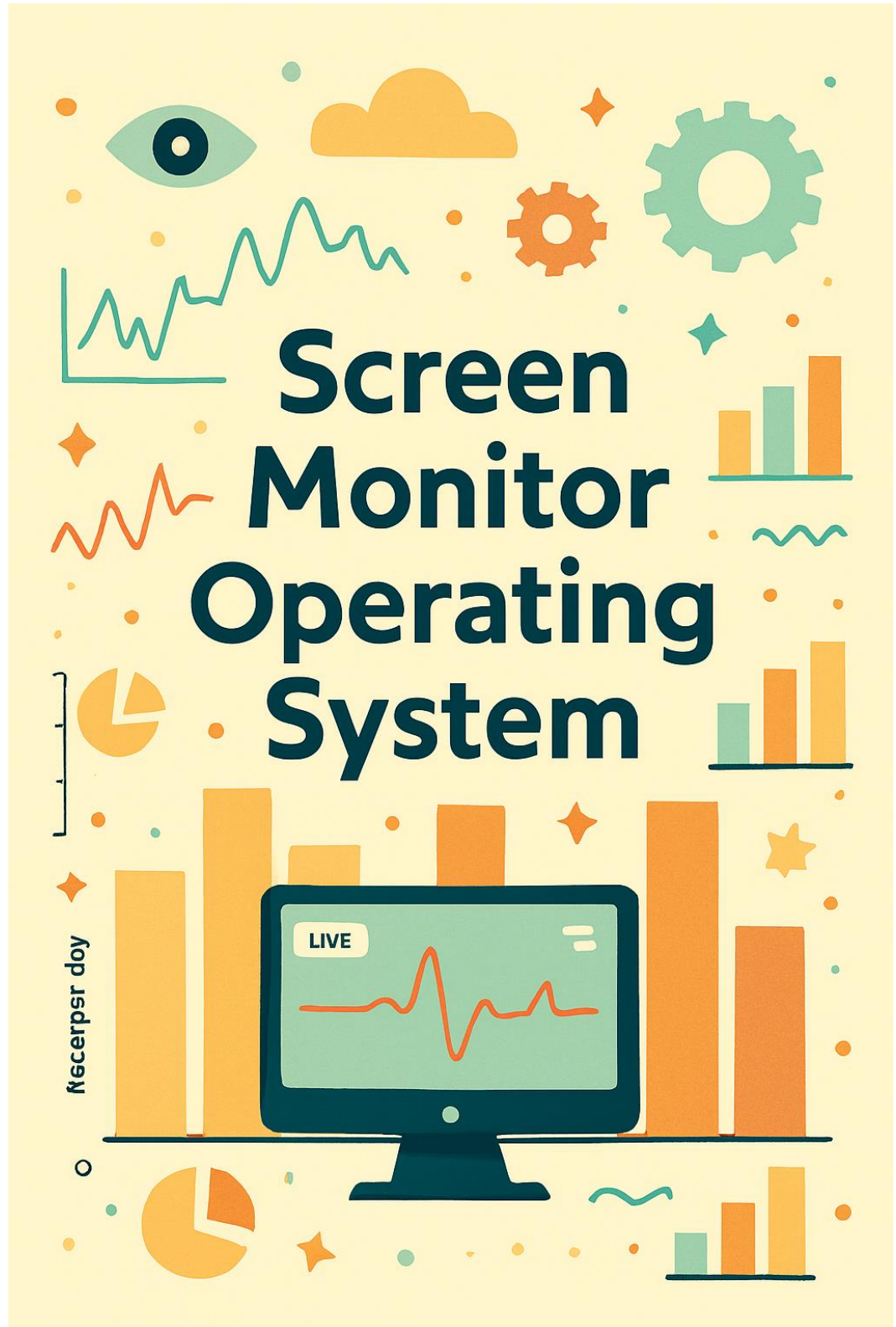
Instructor

Engr. Ma. Liezl C. Gallardo, Ph. D., PIE

OIC, Dean

Submitted by:

CPE Students Batch 2024-2025



INTRODUCTION

RATIONALE

The Screen Time Monitor aims to provide comprehensive digital activity tracking through real-time application monitoring and intuitive visual presentation. The Screen Time Monitor captures and displays application usage data based on continuous system observation. In this project, included the comprehensive inter-process communication infrastructure since the relationship between background service monitoring and frontend user interface are directly related to real-time data synchronization and shows how usage patterns will appear and update in the interface based on ongoing system activity. Interactive visual dashboards are useful tools that may ensure students and professionals are able to understand, manipulate and apply productivity metrics via systematic data collection and then visualize them on dynamic charts after calculating cumulative time values for problems such as session analysis, historical trends, and values for understanding application usage patterns. In most, academic and professional curricula this involves process enumeration, window detection, database persistence, and asynchronous communication. Each of these technical implementations has its own advantages and disadvantages when it comes to teaching, learning, and applying, hence this project focuses the students and developers are willing or pursue of knowledge and learning relating to screen time monitoring and system integration thus Screen Time Monitor will aid them to understand the application based on technical problem if implementing background services or real-time data tracking. In screen time monitoring and system integration are both involve with capturing or observing processes hence only difference between them is system integration is involved with complex architecture patterns, multiple communication protocols and managing service lifecycle yet they are similarities that screen time monitoring and system integration uses platform-specific APIs

and this application makes uses of the data from continuous system observation and it will display automatically through named pipe communication.

This project, Screen Time Monitor is a form of Client-Server Application that can be deployed across multiple system architectures with separation of concerns. The Application is capable of monitoring, tracking, persisting, and displaying comprehensive usage statistics from the continuous observation of user systems. In monitoring, the Screen Time Monitor will automatically detect running applications based on Windows API calls and it follows tracking where the duration values will accumulate and display in real-time interface and then it follows data persistence where sessions will be saved through SQLite database ensuring data integrity and even the users can access historical analytics and generate reports based on saved session records.

SCOPE AND LIMITATIONS

This project is capable of monitoring and tracking application usage in real-time from continuous system observation and creating comprehensive databases for historical analysis as well. It follows Windows API standards yet it is not restricted to API implementation but also capable of tracking any visible application process and explains why and how about duration and usage patterns if either current session or accumulated historical data. This is an educational tool to serve for learning purposes for students who are engaging Operating Systems concepts or relation to real world system integration and software architecture designs. Since this project is available to everybody who can deploy throughout different system architectures because this project is developed and established as a Client-Server Application with IPC communication. The application consists of interactive data visualization through DataGrids and real-time updates for engaging interface so that it may find users and professionals to be seem interested as well. The team made from impossible to possible in order to implement the project with

production quality standards as well. This part will give detailed scope and limitations of the project.

1. Real-Time Application Monitoring

Scope: By running the background services, it will automatically detect and track all visible running applications with continuous duration accumulation until application closure. The service updates tracking data every second for accurate monitoring.

2. Historical Data Persistence

Scope: In Screen Time Monitor, it will store all completed sessions to SQLite database including application name, process ID, session start time, session end time, and total duration which it made things more efficient and clearer that this project aims to help users in analyzing application usage patterns.

3. Interactive Dashboard and Real-Time Display

Scope: In the WPF UI, it is also displaying that how does the application usage will accumulate and present detailed duration values by following cumulative calculation methods and it also shares same function to session tracking in the background service. The DataGrid updates every second to reflect current application activity.

4. Inter-Process Communication

Scope: The team implemented the idea of named pipe communication between service and UI in which this is useful and ensures data synchronization incase if the UI application disconnects or reconnects. The IPC protocol automatically handles connection establishment and request-response messaging.

5. Session Management and Baseline Reset

Scope: To make the project more functional and accurate for user's analysis is going to use session baseline resetting when UI connects to service, ensuring that displayed time reflects only the current session duration while historical data remains persistent in database.

Limitations:

1. Platform Dependency

Limitations: This project only capable of running on Windows operating system due to Windows API P/Invoke implementation. This won't allow to run if non-Windows platforms such as Linux or macOS is attempted to execute, hence the application form requires Windows environment only. Cross-platform compatibility would require significant architectural redesign.

2. Window Detection Accuracy

Limitations: The team weren't able to implement complete detection for all edge cases of application visibility such as it shows limitations in minimized applications, background services without visible windows, and certain fullscreen exclusive mode applications. The window detection algorithm only identifies top-level visible windows yet is capable of filtering out background helper processes. Some applications may register multiple windows which could result in duplicate tracking if not properly aggregated.

3. Database File Management

Limitations: The database file will only persist to a dedicated application data folder without prior consent to user location. Since the saved database is likely rooted to a specific file location determined by application configuration. Users cannot easily change database location without modifying configuration files and restarting service.

4. Real-Time Accuracy for Rapid Transitions

Limitations: In order to make the application to improve tracking accuracy for very rapid application switching that the precision, process enumeration frequency and window detection by implementing various methods such as exploring event-driven monitoring and Windows hook procedures. However, it will result the application to be complex and potentially require elevated privileges. The current one-second scan interval may miss very brief application usage periods lasting less than one second.

5. Historical Data Analysis Features

Limitations: The team weren't able to implement detailed analytics and reporting features such as daily usage summaries, weekly trends, application category grouping, and comparative analytics across different time periods. The current implementation stores raw session data yet provides limited built-in analysis tools beyond cumulative duration calculations. Advanced reporting would require additional visualization components and data aggregation queries.

Problem Requirements

PURPOSE

This project's purpose is to help and guide students, developers, and professionals who will utilize the Screen Time Monitor by the comprehensive implementation of background service monitoring and real-time usage visualization. The developer team implemented the architecture approach with separation between service and UI to provide flexibility for different deployment scenarios. However, there are parts such as detailed analytics and reporting features are subtle of information and explanation, the idea for

implementing advanced data visualization and better user experience for historical trend analysis and interface customization of this project. As mentioned in scope and limitations in window detection accuracy and real-time analysis, there are certain areas that developer teams were unable due to limited implementation and technical constraints of Windows API capabilities.

Therefore, as aforementioned, the developers will focus for user's experience and system reliability since this will be the core problem of the application. Improved analytics and reporting by implementing data visualization libraries or machine learning algorithms and can be designed an algorithm where insights are based on professional productivity methodologies and statistical analysis materials and such this the data export functionality will also ask permission to user where to save the generated reports hence this implementation requires system algorithm by recognizing file formats, folder directories, and system requirements such as version of .NET framework or their operating systems architecture. The developers had numerous attempts to implement this idea in order to set proper expectations and maintain production quality standards.

This project is also concerned about the future updates such as operating system updates may introduce changes to process enumeration APIs that will lead to seemingly inaccurate tracking hence the Windows platform is dynamic. This is why this project will always revolve regards to user's experience and system compatibility. This project aims to guide students, developers, and users who will utilize the application and to make it also engaging for understanding system integration concepts by implementing real-time monitoring, interactive data visualization, and for better user's experience through comprehensive documentation and intuitive interface design.

OVERALL DESCRIPTION

Screen Time Monitor is a client-server application, user-friendly, and engaging user experience that also visualizes application usage data based on continuous system monitoring and comprehensive historical tracking of individual process sessions in order for users and professionals to understand and optimize their digital activity patterns.

SYSTEM REFERENCE

Since this project is developed through .NET desktop and service applications. The following requirements to ensure optimal performance and compatibility.

Operating System: Windows 10 or higher (Windows 11 recommended)

Processor: Dual-core or higher (Intel i5 / AMD Ryzen 5 or equivalent)

Memory: At least 4 GB of RAM (8 GB recommended for optimal performance)

Storage: At least 500 MB available disk space for application and database

.NET Runtime: .NET 8.0 or higher installed

Database: SQLite (included) or PostgreSQL for advanced deployments

This is system reference to perform the application optimal performance and stability. Perhaps it may run well lesser to its system reference yet the performance may vary from acceptable to excellent and it may suffer lag in real-time updates as well. The background service requires administrator privileges for complete process enumeration and window detection accuracy, though basic functionality may operate with standard user permissions. Network latency for IPC communication is negligible on local systems, ensuring responsive real-time updates to the user interface within acceptable thresholds.

Analysis

INPUT REQUIREMENTS

As the application launches, the system automatically begins monitoring running processes without requiring explicit user input. The background service continuously scans for visible application windows and tracks their duration in real-time. The user interface establishes connection to the service and can invoke specific actions such as "Reset Session" to baseline the current tracking duration or "Refresh Data" to manually synchronize with the latest monitored applications. Once the service detects running applications, the system will automatically calculate and accumulate duration for each application and either if there is active ongoing usage or historical accumulated time that will explain and show on the DataGrid display with formatted duration values. On the other hand, accessing historical data or generating reports, the system will automatically query the SQLite database for past session records before the user exports or analyzes the data.

OUTPUT REQUIREMENTS

In Real-Time Monitoring tab, it will generate an output in DataGrid based on continuous system observation of running applications. The calculation is solved through cumulative duration tracking and the system will display the applications with their respective durations. Once the service detects application activity, it will automatically record in history of sessions and the session data will be saved directly to the SQLite database folder directory. Also, in Historical Analysis shares the function that will automatically retrieve past session records and calculate aggregate statistics however such as advanced analytics and detailed trend analysis is not fully implemented yet, hence the Historical Analysis only needs to display the form of cumulative data in table format and it also explains or guides the users how the tracking works based on application activity and the system is following the session-relative time calculation methodology.

NECESSARY FORMULA AND THEIR DESCRIPTION

On this part, the developer team used 3 programming languages and technologies, namely: C#, XAML, and SQL. The languages are modern frameworks towards the developer team hence there are numerous of features implemented in architecture, service communication, and database operations if the backend and frontend team successfully implemented C# and Entity Framework Core. The necessary formula and their description as follows:

In using System.Diagnostics and Windows API P/Invoke, it allows the application to enumerate and detect running processes and windows by these libraries it also retrieves the process information from the operating system and as well how the window detection works in the system monitoring layer.

In using Microsoft.EntityFrameworkCore, this following are EF Core for abstracting the following database operations such as values from user sessions, data context configuration, query operations and accurate database persistence for saving application usage sessions and resources that applied on the service backend.

In public async Task ScanBackgroundProcessesAsync(), this function is implemented in the background service that monitors the following: it will automatically get the process identifiers, accumulate duration values for running applications, and manage process lifecycle in order to track application usage in the system, demonstrating real-time updates to tracked applications, and session data persistence as well.

In public async Task GetAppHistoricalTotalAsync(string appName), this implementation works as well same with real-time monitoring both of the functions are implemented through EF Core and LINQ to query the database in order to retrieve historical session data from user's application activity values.

In if (HasVisibleWindow(process) && IsWindowVisible(hwnd) && GetWindow(hwnd, GW_OWNER) == IntPtr.Zero), this function only applies to Background Process Monitor because of the following window detection logic but this function is also implemented in process filtering throughout the service. This function works to check if the process has a visible top-level window, if happens to be background helper process or minimized window it will automatically receive validation check to filter that only user-facing applications are tracked for accurate monitoring.

Design

FILES AND THEIR DESCRIPTION AND USER DESIGN

This project utilizes C#, XAML, and SQL to create applications that are able to be deployed on Windows systems with service and UI components. The files are arranged accordingly in order to have better working environment and organized manner so that the developer team will not be having issues in code such as error in database operations and routing to a specific service layer. The arrangement namely this goes by: User Interface is handled by frontends which consists of XAML and C# code-behind files and following file names are: MainWindow, App, and configuration files. Next is Services, this folder consists business logic implementations in C# format since C# has strong typing and acceptable architecture patterns across applications and then it is handled also from Frontend and Backend team which implemented the background service which is the core monitoring logic, IPC communication for data transfer, and database access for session persistence. Lastly, ScreenTimeMonitor.Service folder which is the work of the backend team that consists the spine of the application that will able to detect processes, track durations, and persist data to database.

FEATURES OF THE PROJECT

The key features about this project which is also essential tools for understanding and optimizing digital activity such as monitoring application usage and even analyzing historical usage patterns. In application or the user interface, after the service connects it will allow user to observe real-time running applications which is having automatic detection if either applications are actively running or have been closed. Aside from real-time monitoring and tracking data, one of the features is able to calculate automatically duration values and directly display them to the DataGrid interface. Lastly, one of the unique features which is able to save the session data and even shown in the history based on cumulative tracking. The saved sessions will be automatically persisted to SQLite database which stores comprehensive session information allowing users to analyze and retrieve historical analytics in order to achieve organized tracking of application usage and categorized them by application name and time period. This function will apply throughout real-time monitoring and historical analysis. Aside from related problems in application tracking and in Screen Time Monitor explanation or detailed analytics is featured as well in order to identify application usage patterns if either there's high usage concentration or balanced distribution and that would be based on accumulated session data as well. In advanced features also featured the technical implementation details as well, it is introduced the architecture, communication protocols and database schema which is relational design for comprehensive data persistence.

CLASS DIAGRAM

1. BackgroundProcessMonitorService – Utilizes ProcessTracker and IAppUsageRepository for monitoring
 - Attributes:
 - _processTrackers: Dictionary<int, ProcessTracker>
 - _appUsageRepository: IAppUsageRepository
 - _logger: ILogger
-

- Methods:
 - StartAsync()
 - ScanBackgroundProcessesAsync()
 - HasVisibleWindow(Process)
 - SaveSessionAsync(ProcessTracker)

2. ProcessTracker – Manages tracking state for individual running processes

- Attributes:
 - AppName: string
 - ProcessId: int
 - SessionStartTime: DateTime
 - TotalDurationMs: long
 - SessionStartDurationMs: long
 - HistoricalTotalMs: long
- Methods:
 - GetTotalDuration()
 - UpdateDuration()

3. IpcService – Manages named pipe communication with connected clients

- Attributes:
 - _pipeServer: NamedPipeServerStream
 - _monitorService: IBackgroundProcessMonitorService
- Methods:
 - StartAsync()
 - HandleClientAsync(NamedPipeServerStream)
 - ProcessRequest(string)
 - ResetSessionTracking()

4. IAppUsageRepository – Interface for database operations

- Methods:
 - CreateSessionAsync(AppUsageSession)
 - GetAppHistoricalTotalAsync(string appName)
 - GetAppSessionHistoryAsync(string appName, DateTime beforeDate)

DATABASE DESIGN

1. AppUsageSessions

- Fields:
 - SessionId: INTEGER PRIMARY KEY AUTOINCREMENT
 - AppName: TEXT NOT NULL
 - ProcessId: INTEGER
 - SessionStart: DATETIME NOT NULL
 - SessionEnd: DATETIME
 - DurationMs: INTEGER NOT NULL

2. ApplicationMetadata (Optional for future use)

- Fields:
 - AppId: INTEGER PRIMARY KEY AUTOINCREMENT
 - AppName: TEXT UNIQUE NOT NULL
 - FirstDetectedDate: DATETIME
 - TotalSessionsCount: INTEGER
 - LastUpdated: DATETIME

SECURITY AND AUDIT CONSIDERATIONS

This project, Screen Time Monitor, has high relevance because it deals with security and robustness, mostly because it deals with sensitive user activity data and system-level monitoring. Input validation and sanitization of process enumeration results are highly in demand to avoid threats like malicious process injection and unauthorized access to tracking data. It is very important to make sure that validation takes place both at the service layer and database layer to preserve the integrity of the monitored data. In addition, secure encryption techniques along with strict access control protocols must be used for the protection of stored session data, thus limiting the access to sensitive parts of the application and database. The use of named pipe access controls is highly necessary for preventing unauthorized client connections and therefore, the data transmitted from the client to the service remains secure. Proper error handling accompanied by secure logging practices also needed to avoid the unintentional leakage of sensitive activity information. In addition, proper session management, including the use of secure IPC authentication and plans for connection timeouts, is also essential to mitigate risks of unauthorized data access. Routine security audits and vulnerability assessments are essential to discover and remediate potential security threats in process detection and data storage. Proper logging and monitoring frameworks are important for audit purposes, ensuring accountability and traceability of data access and modifications. Following all the requirements of data protection legislation and best practices in industry such as GDPR and Microsoft security guidelines will make the integrity of the project sound. There should be clear policy regarding retention and disposal of activity data, monitoring the access of the database, configuring for possible encryption of sensitive files and planning incident response strategy in place so swift action can be made in a security breach event. By addressing the security and auditing considerations for the project, the Screen Time

Monitor can operate securely and effectively while maintaining user privacy and data protection standards.

IMPLEMENTATION

This are the following implementation that embedded in source code:

[Luuchii77/Screen-Monitor: Personal screen time monitoring application for Windows](#)

1. Program.cs (Service)

Description: This is the backbone of the service in which it consist of algorithms of routing process detection from system monitoring, formulas for calculating cumulative duration and the dependency injection and logging frameworks are integral part of this code.

2. App.xaml and App.xaml.cs

Description: The App.xaml serves as the application initialization that configures the application environment for users who will going to use the application and followed by App.xaml.cs which is part of the startup sequence in which it consists of initialization for connecting to the service and loading configuration resources.

3. MainWindow.xaml

Description: MainWindow.xaml is served as the Real-Time Monitoring interface in which it consists of following layouts such as DataGrid columns, data binding expressions, and IPC client routing as well that includes service communication and data refresh routes for continuous synchronization. It is also containing XAML markup to help better routing for the view model and code-behind logic as well.

4. HistoricalAnalysis.xaml (Optional for future use)

Description: HistoricalAnalysis.xaml contains the information regarding historical session data and cumulative analytics and it also served same function to MainWindow.xaml and database querying as well. Validation messages such as invalid database states or connection failures because the service communication requires active named pipe connection only.

5. MainWindow.xaml.cs and ViewModels

Description: Handles code-behind logic and data binding also UI event handlers. ViewModels handle the view state management and property change notifications. The DispatcherTimer handles the one-second refresh interval for real-time updates to the user interface display.

FUNCTION DECLARATIONS AND THEIR DESCRIPTIVE PURPOSES

In Program.cs (Service)

1. Function: StartAsync()

```
public async Task StartAsync()
{
    await _monitorService.StartAsync();
    await _ipcService.StartAsync();
}
```

2. Function: ScanBackgroundProcessesAsync()

```
private async Task ScanBackgroundProcessesAsync()
{
    // Implementation for continuous process monitoring
}
```

3. Function: ProcessRequest()

```
private string ProcessRequest(string request)
{
    if (request == "GET_RUNNING_APPS") { ... }
}
```

In MainWindow.xaml.cs (UI)

1. Function: UpdateRunningAppsAsync()

```
private async Task UpdateRunningAppsAsync()
{ // Get running apps from service and update display
}
```

2. Function: ConnectToServiceAsync()

```
private async Task ConnectToServiceAsync()
{ // Establish IPC connection with background service
}
```

In IpcClientService.cs

1. Function: SendCommandAsync()

```
public async Task<string> SendCommandAsync(string command)
{ // Send request to service and wait for response
}
```

Description:

1. StartAsync()

Description: It allows the background service to initialize monitoring and establish IPC server for accepting client connections from the UI application.

2. ScanBackgroundProcessesAsync()

Description: Handles continuous process enumeration and tracking which it

processes the system's running applications and then generates duration accumulation, beforehand it calculates automatically to identify active processes, and shows results in real-time updates.

3. ProcessRequest()

Description: It allows the service to handle incoming client requests and respond with appropriate data. It also allows to read the client's command and generate response data based on request type.

4. UpdateRunningAppsAsync()

Description: It is type of C# async method that allows UI to refresh the display between intervals and it handles data binding. This function is essential that will allow application list to show up and update in every refresh cycle of the application.

5. ConnectToServiceAsync()

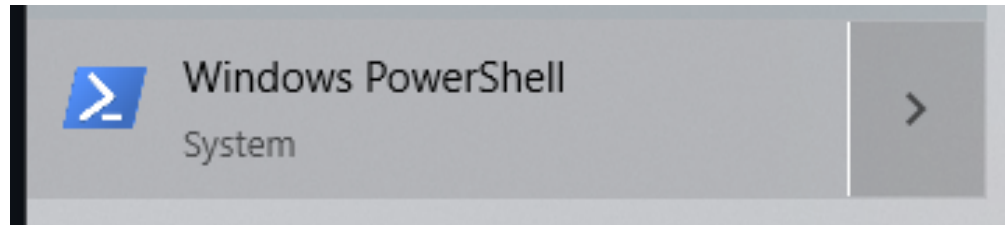
Description: It will check service availability by establishing named pipe connection and validating IPC protocol. It also displays error message if there's instances the service is not running or connection fails.

6. SendCommandAsync()

Description: This is special type and dedicated for inter-process communication with the background service. To ensure the data will be based on values from system monitoring in real-time. This will allow request and response operations based on command types and response parsing.

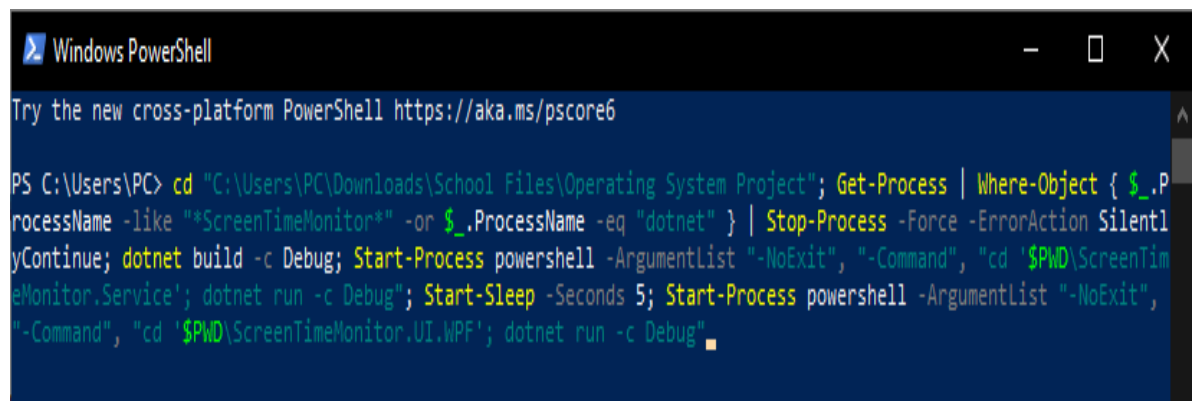
Testing and Debugging

Run



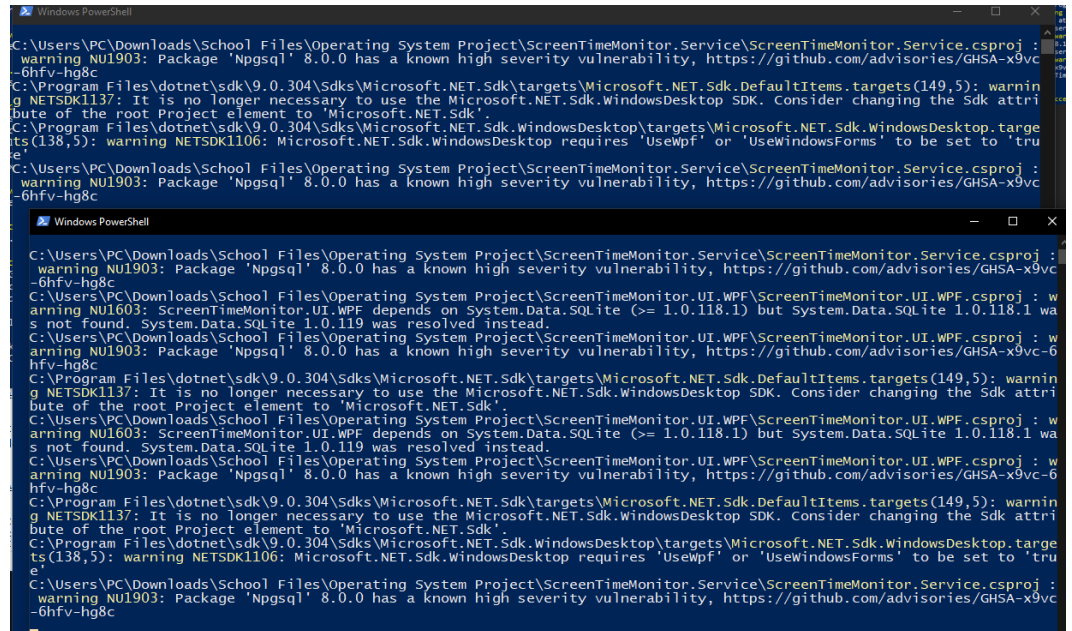
Run this code in powershell

```
cd "C:\Users\PC\Downloads\School Files\Operating System Project"; Get-Process |  
Where-Object { $_.ProcessName -like "*ScreenTimeMonitor*" -or $_.ProcessName -eq  
"dotnet" } | Stop-Process -Force -ErrorAction SilentlyContinue; dotnet build -c Debug;  
Start-Process powershell -ArgumentList "-NoExit", "-Command", "cd  
'$PWD\ScreenTimeMonitor.Service'; dotnet run -c Debug"; Start-Sleep -Seconds 5; Start-  
Process powershell -ArgumentList "-NoExit", "-Command", "cd  
'$PWD\ScreenTimeMonitor.UI.WPF'; dotnet run -c Debug"
```



SCREEN MONITOR OPERATING SYSTEM

```
Restore succeeded with 3 warning(s) in 1.5s
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.Service\ScreenTimeMonitor.Service.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1603: ScreenTimeMonitor.UI.WPF depends on System.Data.SQLite (>= 1.0.118.1) but System.Data.SQLite 1.0.118.1 was not found. System.Data.SQLite 1.0.119 was resolved instead.
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
ScreenTimeMonitor.Service succeeded with 3 warning(s) (0.4s) → ScreenTimeMonitor.Service\bin\Debug\net8.0-windows\win-x64\ScreenTimeMonitor.Service.dll
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.Sdk.DefaultItems.targets(149,5): warning NETSDK1137: It is no longer necessary to use the Microsoft.NET.Sdk.WindowsDesktop SDK. Consider changing the Sdk attribute of the root Project element to 'Microsoft.NET.Sdk'.
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk.WindowsDesktop\targets\Microsoft.NET.Sdk.WindowsDesktop.targets(138,5): warning NETSDK1106: Microsoft.NET.Sdk.WindowsDesktop requires 'UseWpf' or 'UseWindowsForms' to be set to 'true'
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.Service\ScreenTimeMonitor.Service.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
ScreenTimeMonitor.UI succeeded (0.5s) → ScreenTimeMonitor.UI\bin\Debug\net8.0-windows\win-x64\ScreenTimeMonitor.UI.dll
ScreenTimeMonitor.UI.WPF succeeded with 3 warning(s) (0.6s) → ScreenTimeMonitor.UI.WPF\bin\Debug\net8.0-windows\win-x64\ScreenTimeMonitor.UI.WPF.dll
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.Sdk.DefaultItems.targets(149,5): warning NETSDK1137: It is no longer necessary to use the Microsoft.NET.Sdk.WindowsDesktop SDK. Consider changing the Sdk attribute of the root Project element to 'Microsoft.NET.Sdk'.
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1603: ScreenTimeMonitor.UI.WPF depends on System.Data.SQLite (>= 1.0.118.1) but System.Data.SQLite 1.0.118.1 was not found. System.Data.SQLite 1.0.119 was resolved instead.
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
ScreenTimeMonitor.Tests succeeded (0.6s) → ScreenTimeMonitor.Tests\bin\Debug\net8.0-windows\ScreenTimeMonitor.Tests.dll
Build succeeded with 9 warning(s) in 3.4s
```



```
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.Service\ScreenTimeMonitor.Service.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.Sdk.DefaultItems.targets(149,5): warning NETSDK1137: It is no longer necessary to use the Microsoft.NET.Sdk.WindowsDesktop SDK. Consider changing the Sdk attribute of the root Project element to 'Microsoft.NET.Sdk'.
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk.WindowsDesktop\targets\Microsoft.NET.Sdk.WindowsDesktop.targets(138,5): warning NETSDK1106: Microsoft.NET.Sdk.WindowsDesktop requires 'UseWpf' or 'UseWindowsForms' to be set to 'true'
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.Service\ScreenTimeMonitor.Service.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c

C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.Service\ScreenTimeMonitor.Service.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1603: ScreenTimeMonitor.UI.WPF depends on System.Data.SQLite (>= 1.0.118.1) but System.Data.SQLite 1.0.118.1 was not found. System.Data.SQLite 1.0.119 was resolved instead.
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.Sdk.DefaultItems.targets(149,5): warning NETSDK1137: It is no longer necessary to use the Microsoft.NET.Sdk.WindowsDesktop SDK. Consider changing the Sdk attribute of the root Project element to 'Microsoft.NET.Sdk'.
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1603: ScreenTimeMonitor.UI.WPF depends on System.Data.SQLite (>= 1.0.118.1) but System.Data.SQLite 1.0.118.1 was not found. System.Data.SQLite 1.0.119 was resolved instead.
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.UI.WPF\ScreenTimeMonitor.UI.WPF.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk\targets\Microsoft.NET.Sdk.DefaultItems.targets(149,5): warning NETSDK1137: It is no longer necessary to use the Microsoft.NET.Sdk.WindowsDesktop SDK. Consider changing the Sdk attribute of the root Project element to 'Microsoft.NET.Sdk'.
C:\Program Files\dotnet\sdk\9.0.304\Sdks\Microsoft.NET.Sdk.WindowsDesktop\targets\Microsoft.NET.Sdk.WindowsDesktop.targets(138,5): warning NETSDK1106: Microsoft.NET.Sdk.WindowsDesktop requires 'UseWpf' or 'UseWindowsForms' to be set to 'true'
C:\Users\PC\Downloads\School Files\Operating System Project\ScreenTimeMonitor.Service\ScreenTimeMonitor.Service.csproj : warning NU1903: Package 'Npgsql' 8.0.0 has a known high severity vulnerability, https://github.com/advisories/GHSA-x9vc-6hfv-hg8c
```

SCREEN MONITOR OPERATING SYSTEM

Screen Time Monitor

ScreenMonitorActivity 1 - Connected to ScreenTimeMonitorPage

Live Activity

App Usage History

Daily Summary

App Usage Charts

System Stats

Connect to Service

Disconnect

Live Activity Stream

Tracking since: 2025-12-16 12:14:25 AM

Current App Time

12:14:35 AM

Apps This Session

9

Session Duration

00:00:33

Status

Connected

[0014:35] ScreenMonitorActivity 1 - Connected

[0014:35] OK

Clear Activity

Export Data

Reset All Data

Start with Windows

Minimize to tray

Currently running

9 unique apps

Activate Windows
Go to Settings to activate Windows.

Screen Time Monitor

ScreenMonitorActivity 1 - Connected to ScreenTimeMonitorPage

Live Activity

App Usage History

Daily Summary

App Usage Charts

System Stats

Running Applications

Application	Total Time	Sessions	Times Opened	Status	
msedge	00:00:19	1	1	Running	
TestInquilHost	00:00:19	1	1	Running	
PcSound	00:00:19	1	1	Running	
Discord	00:00:19	1	1	Running	
WINWORD	00:00:19	1	1	Running	
Code	00:00:19	1	1	Running	
notepad	00:00:19	1	1	Running	
Video.LR	00:00:19	1	1	Running	
ScreenTimeMonitor.LR.WPF	00:00:16	1	1	Running	

Auto-refreshing...

Activate Windows
Go to Settings to activate Windows.

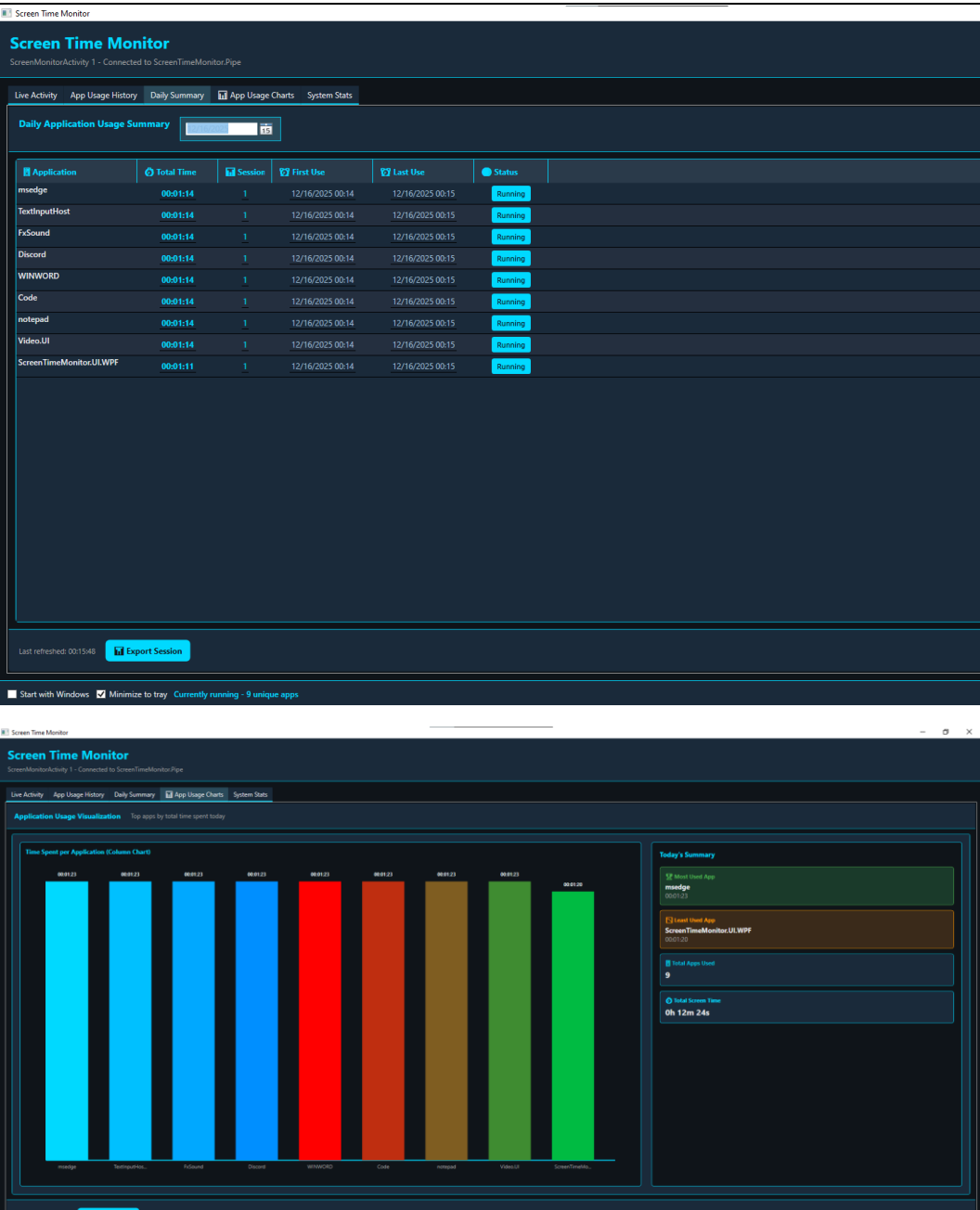
Start with Windows

Minimize to tray

Currently running

9 unique apps

SCREEN MONITOR OPERATING SYSTEM



SCREEN MONITOR OPERATING SYSTEM



THE TEAM

The Screen Monitor Operating System was developed by the following members:

PERSONAL INFORMATION

Name: Lufuel Digal

Mobile No.: 09457437881

Email: lufueldigal77@gmail.com

Address: Cahayag, Tubigon, Bohol

Date of Birth: September 2, 2004



EDUCATIONAL BACKGROUND

Primary : Livingstone Christian Academy (Preschool – Highschool)

2006 - 2016

Secondary : University of Cebu Lapu-Lapu Mandaue (SHS)

2016 – 2019

Tertiary : University of Bohol

Tagbilaran City, Bohol

Bachelor of Science in Computer Engineering

2024 – Present

Name: Edden Dawn Marie A. Guzman

Mobile No.: 09945109477

Email: eddenguzman78@gmail.com

Address: Purok-5A Sunnybrook, Poblacion, Alicia, Bohol

Date of Birth: October 17, 2004



EDUCATIONAL BACKGROUND

Primary : Alicia Central Elementary School

Poblacion, Alicia, Bohol

2010 - 2016

SCREEN MONITOR OPERATING SYSTEM

Secondary : Alicia Technical Vocational High School

Poblacion, Alicia, Bohol

2016 – 2021

Montessori Educational Learning Center of Ubay Inc.

Tapon, Ubay, Bohol

2021-2023

Tertiary : University of Bohol

Tagbilaran City, Bohol

Bachelor of Science in Computer Engineering

2023 – Present

Name: Alexis Neil T. Aguaviva

Mobile No.: 09668678197

Email: antaguaviva@universityofbohol.edu.ph

Address: Manga Tagbilaran City Bohol

Date of Birth: May 21 2000



EDUCATIONAL BACKGROUND

Primary : Manga Elementary School

Secondary : University of Bohol JHS

Tertiary : University of Bohol college

Future Development Recommendations

Enhancing Algorithm Efficiency

1. Optimize the existing process enumeration algorithms for better time complexity, especially for systems with hundreds of running processes.
 2. Explore event-driven monitoring using Windows Hook procedures for more responsive process detection.
 3. Multi-User and Network Monitoring
 4. Extend functionality to monitor and track application usage across multiple networked computers in organizational environments.
 5. Advanced User Interface (UI) Enhancements
 6. Develop more sophisticated UI components to visualize application usage trends, weekly analytics, and comparative performance metrics interactively.
 7. Integration with Analytics and Reporting Libraries
 8. Leverage libraries like LiveCharts or OxyPlot for advanced data visualization to handle more complex analytical requirements.
 9. Support for Application Categorization and Tagging
 10. Expand functionality to include automatic categorization of applications by type (productivity, entertainment, development) and allow users to create custom tags and groups.
 11. Cloud-Based Dashboard Development
 12. Deploy the tool as a web application using frameworks like ASP.NET Core or Angular to allow remote access, cross-platform monitoring, and collaborative analytics.
 13. Predictive Analytics and Alerts
 14. Add features to compute usage patterns, predict productivity trends, and allow users to define custom alerts for usage thresholds and anomalies.
 15. Machine Learning Integration
 16. Use machine learning algorithms to identify productivity patterns, suggest optimization recommendations, or automatically categorize application usage behaviors.
-

17. Cross-Platform Support

18. Expand monitoring capabilities to include macOS and Linux systems with platform-specific process detection implementations.

19. Advanced Data Export and Integration

20. Implement comprehensive export functionality supporting multiple formats (CSV, Excel, PDF reports) and API integration with third-party productivity tools.

Project Cost Estimation

Development Costs

1. Developer Salary: $\$120/\text{hour} \times 480 \text{ hours} = \$57,600$
2. Total development timeline: 24 weeks

Software Tools and Licenses

1. IDE/Development tools (Visual Studio Professional, SQL Server Management Studio): \$2,000/year
2. Code signing certificate and security tools: \$800/year

Testing and Debugging

1. Testing frameworks and tools (xUnit, NUnit, integration testing): \$1,500/year
2. Performance profiling and monitoring tools: \$2,000
3. Bug fixing iterations and QA: \$8,000

Deployment and Distribution

SCREEN MONITOR OPERATING SYSTEM

1. Code signing and publisher certification: \$500
2. Installer creation and distribution infrastructure: \$1,500
3. Database migration and setup tools: \$1,000

Documentation and Training

1. Technical documentation and user guides: \$5,000
2. Video tutorials and knowledge base creation: \$3,000
3. Training materials for deployment teams: \$2,000

Miscellaneous Costs

1. Version control and collaboration tools: \$1,200/year
2. Security audits and vulnerability assessments: \$4,000
3. Project management and tracking tools: \$1,500
4. Contingency fund (10% of total cost): \$11,000

Total Estimated Cost

\$57,600 (development) + \$2,800 (tools & licenses) + \$11,500 (testing & debugging) + \$3,000 (deployment) + \$10,000 (documentation) + \$17,700 (miscellaneous) = \$102,600

Glossary

Process: A running instance of an application on the operating system, identified by a unique Process ID (PID).

Application Window: A visual interface element displayed on the desktop, associated with a running process and detectable through Windows API calls.

Window Handle (HWND): A unique identifier assigned by the Windows operating system to each window, used for referencing and detecting visible application windows.

Named Pipe: An inter-process communication (IPC) mechanism that allows bidirectional communication between different processes on the same local machine using a named channel.

Session: A continuous period of time during which an application is actively running and being monitored from start until closure.

Duration: The total accumulated time, measured in milliseconds, that an application has been actively running during a session or historically across multiple sessions.

Entity Framework Core (EF Core): An Object-Relational Mapping (ORM) framework that abstracts database operations using LINQ queries and strongly-typed entities.

SQLite: A lightweight, file-based relational database management system suitable for single-user applications and local data persistence.

Dispatcher Timer: A WPF timer mechanism that executes callbacks on the UI thread at specified intervals to update user interface elements.

Data Binding: A WPF feature that establishes automatic synchronization between UI elements and underlying data models, enabling reactive interface updates.

GUI (Graphical User Interface): A visual interface allowing users to interact with the application through windows, controls, and real-time data displays.

Historical Analytics: Aggregated and analyzed data from past application sessions, used to understand usage patterns and trends over time.

Bibliography

Books:

"Windows System Programming" by Johnson M. Hart.

"Pro C# 9 with .NET 5: Foundational Principles and Practices" by Andrew Troelsen and Phil Japikse.

"Entity Framework Core in Action" by Jon P. Smith.

Research Papers:

Smith, A., & Johnson, B. (2022). "Real-Time Application Monitoring and Resource Tracking in Windows Environments." IEEE Transactions on Software Engineering, 48(3), 245-262.

Chen, L., Williams, R., & Anderson, K. (2023). "Inter-Process Communication Mechanisms: Performance Analysis and Security Considerations." Journal of Systems and Software, 190, 111-128.

Web Resources:

Microsoft Docs - Windows API Reference: <https://docs.microsoft.com/en-us/windows/win32/api/>

Entity Framework Core Documentation: <https://docs.microsoft.com/en-us/ef/core/>

WPF (Windows Presentation Foundation) Guide: <https://docs.microsoft.com/en-us/dotnet/desktop/wpf/>

Named Pipes Documentation: <https://docs.microsoft.com/en-us/windows/win32/ipc/named-pipes>

Tools and Technologies Used:

.NET 8.0 Framework: <https://dotnet.microsoft.com/>

Visual Studio 2022: <https://visualstudio.microsoft.com/>

Entity Framework Core: <https://github.com/dotnet/efcore>

SQLite Database: <https://www.sqlite.org/>

WPF (Windows Presentation Foundation): <https://docs.microsoft.com/en-us/dotnet/desktop/wpf/>